

3. Brainstorming, Idea Generation & Prioritization Template

Ideation Phase: Brainstorm & Idea Prioritization

Header	Information
Date	02 October 2026
Team ID	
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Step 1: Team Gathering, Collaboration and Select the Problem Statement

- **Selected Problem Statement: PS-1 & PS-2** — Restricting record viewing, creation, modification, and deletion based on dynamic record field values and specialized roles using Advanced ACL scripts in ServiceNow.

Step 2: Brainstorm, Idea Listing and Grouping

Idea Listing (Raw Ideas Generated):

1. **Idea A:** Create separate tables for each department/branch to isolate data completely.
2. **Idea B:** Use Before Query Business Rules to filter out records from list queries automatically.
3. **Idea C:** Implement Advanced Script-Controlled ACLs evaluating current.branch field values along with user roles (gs.hasRole()).
4. **Idea D:** Use Client Scripts and UI Policies to hide fields on forms.
5. **Idea E:** Assign granular role hierarchy (bb1 for Read, bb2 for Create, bb3 for Write, bb4 for Delete) coupled with dynamic script evaluation.

Idea Grouping & Categorization:

- **Category 1: UI-Level Hiding (Client-Side)**
 - *Ideas:* Idea D (Client Scripts / UI Policies)
 - *Assessment:* Insecure; leaves data vulnerable to direct API queries or URL hacks.
- **Category 2: Database Schema Separation**
 - *Ideas:* Idea A (Separate Tables per Department)
 - *Assessment:* Unscalable; increases database overhead and maintenance complexity.
- **Category 3: Server-Side Query / Rule Control**
 - *Ideas:* Idea B (Business Rules), Idea C (Script-Controlled ACLs), Idea E (Granular CRUD Role ACLs)
 - *Assessment:* Highly secure, enforced directly by the ServiceNow security engine at runtime.

Step 3: Idea Prioritization Matrix

Idea Description	Feasibility (High/Med/Low)	Security Value (High/Med/Low)	Impact	Priority Rank
Idea C & E: Script-Controlled ACLs for CRUD Operations (u_institution_details table with roles bb1–bb4)	High	High	Enforces complete server-side security across forms, lists, and APIs with minimal execution latency (<1.8ms).	Rank 1 (Selected)
Idea B: Before Query Business Rules	High	Medium	Good for list filtering, but doesn't natively control individual field write/delete operations as effectively as ACLs.	Rank 2
Idea A: Separate Tables per Branch	Low	Medium	Creates redundant architecture and high administrative overhead.	Rank 3
Idea D: Client Scripts / UI Policies	High	Low	Does not provide security; only affects UI visibility.	Rank 4

Final Solution Summary:

Implement **Script-Controlled ACLs** on u_institution_details table:

- **READ ACL:** Requires role bb1 + script check (gs.hasRole('admin') OR Branch == 'EEE').
- **CREATE ACL:** Requires role bb2.
- **WRITE ACL:** Requires role bb3.
- **DELETE ACL:** Requires role bb4.